Introduction

CoreRelm is a lightweight, attribute-based ORM for C# that stays close to ADO.NET style development while reducing repetitive data access code.

It is designed for developers who want:

- Strong control over SQL and transactions
- Attribute-driven model mapping
- Predictable behavior without heavy abstraction
- A simple context and dataset workflow
- A MySQL-focused ORM for .NET 8+

CoreRelm favors explicitness over magic. The library gives you structure for models, datasets, options, helpers, and migrations, while still keeping the database visible in the way you work.

What CoreRelm is trying to be

CoreRelm is intended to sit in the space between writing raw ADO.NET everywhere and adopting a heavier ORM abstraction.

At a high level, it provides:

- Attribute-based model metadata
- A `RelmContext`-centered workflow
- `RelmDataSet` access patterns for models
- Options-based context configuration
- Utility helpers for common relational tasks
- Migrations tooling and schema-oriented APIs
- Support for both eager and lazy context initialization patterns

The goal is not to hide the database. The goal is to make common relational work more structured, more readable, and more repeatable.

Who CoreRelm is for

CoreRelm is a strong fit when you want:

- A practical ORM that still feels close to the database
- Clear transaction boundaries
- Attribute-driven mapping instead of large external configuration systems
- A model and dataset workflow that is simple to reason about
- A codebase where explicit behavior is preferred over implicit conventions

It may be especially appealing if you like the directness of lower-level database code but want more consistency and reuse around model definition, querying patterns, relationships, and schema workflows.

Supported platform and provider model

CoreRelm currently targets:

- .NET 8+
- MySQL 8+ only

The current provider story is intentionally focused. The library and documentation should be read with MySQL as the active provider target.

Core building blocks

Most of the library revolves around a few core ideas:

`RelmContext`

`RelmContext` is the main runtime entry point. It represents a configured database context and is the place where connection behavior, transaction behavior, dataset access, and lower-level execution helpers come together.

`RelmDataSet`

A `RelmDataSet` represents a model-oriented access surface for a particular entity type. This is where many common operations such as querying, loading, creating, and referencing related data are expressed.

`RelmModel`

A `RelmModel` represents a mapped entity. Attributes applied to the model and its members describe how the model relates to the database.

`RelmContextOptions` and `RelmContextOptionsBuilder`

Context options define how a CoreRelm context behaves. The options builder is the main way to configure connection details, auto-open behavior, initialization behavior, and related settings.

Design style

CoreRelm follows a few broad design preferences:

- Explicit transaction handling is preferred
- Configuration should be readable and close to usage
- Metadata should live near the model when practical
- Examples should show the shape of the API clearly

- Runtime behavior should be understandable without guessing

That design style is visible in the QuickStart project as well. The examples there are documentation-style examples meant to show what a call may look like. They are not intended to serve as a polished end-to-end sample application.

What you will find in this documentation

This guide is organized to help different kinds of readers move through the library effectively.

If you are new to CoreRelm:

1. Start with Getting Started
2. Review the Core Concepts section
3. Move into the Working with Data section

If you already know what you need:

- Use the guide pages for narrative explanations and recommended patterns
- Use the API reference for type and member details
- Use `examples/CoreRelm.QuickStart` to see documentation-style usage examples

CoreRelm versus EF and Dapper

CoreRelm is not trying to replace every ORM style.

Compared to Entity Framework, CoreRelm generally favors:

- More explicit behavior
- Less hidden runtime convention
- A simpler mental model for developers who want direct control
- A closer relationship to relational and connection-level details

Compared to Dapper, CoreRelm generally offers:

- More built-in structure around models and datasets
- Attribute-based mapping and metadata
- A more unified ORM-style surface for repeated patterns
- Schema- and migrations-related concepts in the same ecosystem

In practice, CoreRelm is best understood as a deliberate tradeoff: more structure than a very thin micro-ORM, but more explicitness than a heavier convention-driven ORM.

Where to go next

The best next step is Getting Started.

That page will walk through:

- how CoreRelm is configured
- how contexts are initialized
- what a minimal usage flow looks like
- where the QuickStart project fits into the documentation

Getting Started

This page is designed to help a new user understand the basic shape of CoreRelm as quickly as possible.

CoreRelm uses a documentation-first QuickStart project. The examples are intended to demonstrate what a call may look like. They are not required to compile or run as-is.

Before you begin

CoreRelm currently supports:

- .NET 8+
- MySQL 8+ only

The QuickStart project and the broader codebase are currently organized around that provider model.

The basic flow

A typical CoreRelm workflow looks like this:

1. Define a model and apply mapping attributes
2. Define a context
3. Build `RelmContextOptions`
4. Create a context
5. Read or write data through the context and its datasets
6. Use explicit transactions when an operation spans multiple writes

Define a model

In order to work with CoreRelm, you need to define models that represent your database entities. Models are classes decorated with attributes that describe how they map to database tables and columns.

To automatically include the standard metadata columns, your model should inherit from `RelmModel`. This base class includes properties for `Id`, `InternalId`, `CreateDate`, and `LastUpdated`. If you want to define a model without those standard columns, you can inherit directly from `RelmModelClean` and apply the necessary attributes.

The following example shows a simple model with a few properties and a foreign key relationship. The attributes indicate how the model maps to the database, and the navigation properties allow for easy access to related data. This model is *not* migratable as-is because column types and lengths are not specified.

Example model shape

```
[RelmTable("example_models")]
public class ExampleModel : RelmModel
{
    [RelmColumn]
    public string? Name { get; set; }

    [RelmColumn]
    public int Value { get; set; }

    [RelmColumn]
    public string? GroupInternalId { get; set; }

    [RelmForeignKey(foreignKey: nameof(ExampleGroup.InternalId),
localKey: nameof(GroupInternalId))]
    public virtual ExampleGroup Group { get; set; }
}

[RelmTable("example_groups")]
public class ExampleGroup : RelmModel
{
    [RelmColumn]
    public string? Name { get; set; }

    [RelmForeignKey(foreignKey: nameof(ExampleModel.GroupInternalId),
localKey: nameof(InternalId))]
    public virtual ICollection<ExampleModel> Models { get; set; }
}
```

Configure a context

CoreRelm uses `RelmContextOptionsBuilder` to configure runtime behavior.

The QuickStart project shows multiple options builder constructor styles, including:

- starting from defaults
- copying from an existing options object
- initializing from an enum-backed connection reference
- initializing from a connection
- initializing from a connection and transaction
- initializing from a named connection string
- initializing from a raw connection string
- initializing from explicit server, database, user, and password values
- initializing from explicit server, port, database, user, and password values

In practice, most applications should start with the simplest configuration that fits their use case and only override the settings they actually need.

Example maximal options shape

```
var options = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoOpenConnection(true)
    .SetAutoOpenTransaction(false)
    .SetAllowUserVariables(false)
    .SetConvertZeroDateTime(false)
    .SetLockWaitTimeoutSeconds(0)
    .SetAutoInitializeDataSets(true)
    .SetAutoVerifyTables(true)
    .BuildOptions();
```

CoreRelm also supports a more lazy-loading style by disabling automatic dataset initialization and automatic table verification.

```
var quickOptions = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoInitializeDataSets(false)
    .SetAutoVerifyTables(false)
    .BuildOptions();
```

The eager pattern favors readiness up front. The lazy pattern favors lower startup work and loads metadata as needed.

Define a context

A context is the main runtime object used to work with CoreRelm.

The QuickStart project includes an `ExampleContext` that inherits from `RelmContext` and exposes model-oriented data access members.

Example context shape

```
public class ExampleContext(RelmContextOptions contextOptions) : RelmContext(contextOptions)
{
    public IRelmDataSet<ExampleModel>? ExampleModels { get; set; }
    public IRelmDataSet<ExampleGroup>? ExampleGroups { get; set; }
}
```

The exact datasets you expose will depend on your application. The important part is that the context becomes the central place where configuration, connection behavior, transactions, and model access

come together.

Initialize and use a context

The QuickStart shows both standard scoped usage and a more lazy-loading quick-context style.

Example usage shape

```
var relmContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

Example scoped transaction pattern

```
using var scopedContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();

scopedContext.BeginTransaction();

try
{
    // Perform database operations here

    scopedContext.CommitTransaction();
}
catch
{
    scopedContext.RollbackTransaction();
    throw;
}
```

CoreRelm is most readable when transaction boundaries are explicit. If an operation spans multiple writes, prefer a clear begin/commit or begin/rollback pattern.

Common feature areas shown in the QuickStart project

The QuickStart project is broad. It currently demonstrates the shape of calls for several public feature areas, including:

- context usage
- connection examples
- identity helpers
- data row, data table, data object, and data list access patterns
- database work helpers
- bulk table writing

- attributes
- foreign keys
- DTOs
- data loaders
- model reset behavior
- model property naming conventions

Treat the QuickStart project as a usage map. It is there to show how features look in code, even when the examples are intentionally lightweight.

Quick Start in 60 Seconds

The following example is intentionally small and focuses on the shape of the workflow.

It will create an `ExampleContext` context that:

- Automatically opens the connection named `ExampleConnectionString` in the `appSettings.json` file.
- Opens the database connection.
- Leaves the database transaction closed.
- Automatically (eager) initializes all data sets.
- Automatically (eager) verifies data sets against database table list.

1. Build options & create context (one step)

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

2. Use the context

```
var lastInsertId = context.GetLastInsertId();

var tableName = RelmHelper.GetDalTable<ExampleModel>();
var internalId = "some-guid-value";
var idFromInternalId = context.GetIdFromInternalId(tableName, internalId);

if (context.ExampleModels == null)
    context.GetDataSet<ExampleModel>();

var exampleActiveModels = context
    .ExampleModels
    .Where(model => model.Active == true && model.Id > 5)
    .OrderByDescending(model => model.CreateDate)
    .Load();
```

This is not meant to be a complete application. It is meant to show the basic shape of a configured context and a few common operations.

Eager versus quick context behavior

The QuickStart distinguishes between two high-level approaches:

Eager context

An eager context initializes datasets withno data retrieval and verifies tables up front.

This can make some later operations faster because metadata work has already been done.

Quick context

A quick context disables automatic dataset initialization and automatic table verification.

This reduces up-front work and allows metadata to load as needed, but the first operation for a given path may do more work.

This distinction is useful when deciding how much startup work you want versus how much lazy runtime work you are comfortable with. For example, a web application may prefer a quick context to minimize startup time, while a background processing application may prefer an eager context to ensure readiness.

Error handling guidance

A good default pattern is:

- start a transaction explicitly when needed
- commit only after all related operations succeed
- roll back on failure
- keep error paths simple and predictable

Example

```
using var relmContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoOpenTransaction(true)
    .Build<ExampleContext>();

try
{
    // Run database work examples
    // Run bulk write examples

    // Transaction will auto-commit when the context is disposed if no errors occur
```

```
}
catch (Exception ex)
{
    relmContext.RollbackTransaction();
    Console.WriteLine($"An error occurred: {ex.Message}");
}
```

Where to go next

After this page, the best next steps are:

1. Review the Core Concepts section to understand `RelmContext`, `RelmDataSet`, `RelmModel`, and attributes
2. Move into Working with Data for CRUD, querying, relationships, DTOs, loaders, and bulk operations
3. Review Transactions, Migrations, and Advanced Topics for reliability and schema workflows
4. Use the API reference when you need type and member details

CoreRelm Mental Model

This page explains the main runtime shape of CoreRelm and how its core pieces fit together.

CoreRelm is designed to make relational work more structured and readable without hiding the database behind a large abstraction layer. Your models, context, datasets, options, helpers, and migrations all work together, but the database is still visible in the way you use the library.

The basic idea

At a high level, a typical CoreRelm workflow looks like this:

1. Define a model and apply mapping attributes
2. Define a context
3. Configure `RelmContextOptions`
4. Build or create a context
5. Work with models through data sets
6. Use explicit transactions when an operation spans multiple writes

That flow is simple on purpose. CoreRelm prefers explicitness over magic and favors a mental model that is easy to reason about.

The main building blocks

Most of the library revolves around a few core pieces:

- `RelmContext`
- `RelmDataSet<T>`
- `RelmModel`
- `RelmContextOptions`
- `RelmContextOptionsBuilder`

`RelmContext`

`RelmContext` is the main runtime entry point. It is where connection behavior, transaction behavior, dataset access, and lower-level execution helpers come together. If you think of CoreRelm as a system, the context is the center of it.

In practice, the context is where you:

- work with configured database access
- begin, commit, and roll back transactions
- expose your model-oriented datasets
- use lower-level helper operations when needed

`RelmDataSet<T>`

A `RelmDataSet<T>` is the model-oriented access surface for a specific entity type. It is where many common operations are expressed, including querying, loading, creating, and referencing related data.

A data set is not just a collection. It is the place where CoreRelm brings together:

- model metadata
- query patterns
- load and save workflows
- relationship access patterns

`RelmModel`

A `RelmModel` represents a mapped entity. Attributes applied to the model and its members describe how the entity relates to the database.

In CoreRelm, models are not just POCOs with no meaning. They carry mapping intent close to the code that uses them.

`RelmContextOptions` and `RelmContextOptionsBuilder`

Context options define how a CoreRelm context behaves. The options builder is the primary way to configure connection details, auto-open behavior, dataset initialization, table verification, and related settings.

This is one of the key places where CoreRelm's design style shows up: configuration stays readable and close to usage.

How these pieces fit together

A simple way to think about the system is:

- the **model** describes the mapped entity
- the **dataset** is the model-specific access surface
- the **context** holds the configured runtime environment
- the **options** control how that environment behaves

That gives you a layered workflow that still feels direct:

- define how the entity maps
- define where the entity is accessed
- configure how the runtime behaves
- perform database operations explicitly

Metadata stays close to the model

One of CoreRelm's design preferences is that metadata should live near the model when practical. That makes it easier to understand how an entity maps to the database by reading the model directly, rather than chasing separate configuration systems.

This is a major reason the library uses attributes heavily.

Explicit transaction boundaries

CoreRelm prefers explicit transaction handling. That preference is visible throughout the guide and the QuickStart examples. When an operation spans multiple writes, a clear begin/commit or begin/rollback pattern is usually the most readable approach.

That philosophy matters because it affects how the library should be read:

- CoreRelm is not trying to make database boundaries disappear
- CoreRelm is trying to make them more understandable

Eager versus lazy context behavior

A useful high-level distinction in CoreRelm is the difference between an eager context and a quick context.

Eager context

An eager context initializes datasets up front and verifies tables before later operations need them. This favors readiness and can reduce later metadata work.

Lazy context

A lazy context disables automatic dataset initialization and automatic table verification. This reduces up-front work and allows metadata to load as needed.

This distinction is important because it changes the runtime feel of the library:

- eager favors readiness
- lazy favors lighter startup work Neither is universally better. The right choice depends on the application shape.

The role of the QuickStart project

The QuickStart project is documentation-first. Its job is to show what a call may look like, not to serve as a polished end-to-end sample application. That matters when reading examples across the docs: many examples are there to explain usage shape, not to be copied as-is into production code.

Use `examples/CoreRelm.QuickStart` as a usage map for how the API looks in practice.

Where to go next

After this page, the best next steps are:

1. Review Models and Attributes
2. Review Contexts and DataSets
3. Review Options and Configuration

Together, those pages explain the main conceptual pieces that sit underneath the rest of the library.

Models and Attributes

This page explains how CoreRelm models are structured and how attributes describe the relationship between your code and the database.

CoreRelm uses attribute-driven model metadata as one of its central design choices. The goal is to keep mapping information near the model so that relational intent remains easy to read.

What a model is in CoreRelm

A `RelmModel` represents a mapped entity. Attributes applied to the model and its members describe how that entity relates to the database.

In practice, this means a model usually does two things at once:

- it represents domain or application data
- it carries mapping metadata that tells CoreRelm how to treat that data

`RelmModel` versus `RelmModelClean`

CoreRelm supports two broad model starting points.

`RelmModel`

Use `RelmModel` when you want the standard metadata fields included automatically.

Your updated Getting Started page calls out that `RelmModel` includes standard metadata columns such as:

- `Id`
- `InternalId`
- `CreateDate`
- `LastUpdated`

This is the most convenient starting point when those default fields match the shape you want.

`RelmModelClean`

Use `RelmModelClean` when you do **not** want those standard fields included automatically and would rather define everything yourself.

This is useful when:

- you need tighter control over the entity shape
- you are mapping to tables that do not follow the default metadata pattern
- you want to opt into only the fields you explicitly define

Common model-level attributes

CoreRelm uses attributes to describe mapping and schema intent. Common examples include:

- `RelmTable`
- `RelmColumn`
- `RelmKey`
- `RelmForeignKey`

The uploaded codebase inventory also shows a broader attribute surface in the API docs, which indicates that attributes are an important part of the public model.

`RelmTable`

`RelmTable` declares the table that a model maps to.

Example

```
[RelmTable("example_models")]
public class ExampleModel : RelmModel
{
}
```

This is the top-level mapping anchor for the entity.

`RelmColumn`

`RelmColumn` marks a property as a mapped column.

Example

```
[RelmColumn]
public string? Name { get; set; }
```

In your updated Getting Started page, `RelmColumn` is used to mark mapped fields such as `Name`, `Value`, and `GroupInternalId`.

At a conceptual level, `RelmColumn` answers the question:

> Which properties should CoreRelm treat as mapped relational values?

`RelmKey`

`RelmKey` identifies key members. Depending on your model strategy, this may be part of the standard metadata you get from `RelmModel`, or it may be something you define more explicitly when working from a cleaner base model.

Key information matters because it affects:

- identity
- lookup behavior
- relationships
- migrations and schema workflows

`RelmForeignKey`

`RelmForeignKey` describes relationship links between models.

Your updated Getting Started example uses `RelmForeignKey` in both directions:

- from `ExampleModel` to `ExampleGroup`
- from `ExampleGroup` to the collection of `ExampleModel` instances

Example

```
[RelmForeignKey(foreignKey: nameof(ExampleGroup.InternalId),
localKey: nameof(GroupInternalId))]
public virtual ExampleGroup Group { get; set; }
```

This style keeps relationship intent readable directly on the model.

Example model shape

The following example reflects the updated shape you provided:

```
[RelmTable("example_models")]
public class ExampleModel : RelmModel
{
    [RelmColumn]
    public string? Name { get; set; }

    [RelmColumn]
    public int Value { get; set; }

    [RelmColumn]
    public string? GroupInternalId { get; set; }
```

```
    [RelmForeignKey(foreignKey: nameof(ExampleGroup.InternalId),
localKey: nameof(GroupInternalId))]
    public virtual ExampleGroup Group { get; set; }
}

[RelmTable("example_groups")]
public class ExampleGroup : RelmModel
{
    [RelmColumn]
    public string? Name { get; set; }

    [RelmForeignKey(foreignKey: nameof(ExampleModel.GroupInternalId),
localKey: nameof(InternalId))]
    public virtual ICollection<ExampleModel> Models { get; set; }
}
```

This model shape demonstrates:

- table mapping
- column mapping
- use of the default metadata base model
- foreign key navigation intent

Migratable versus non-migratable model shapes

One important point from your updated Getting Started page is that a model may be valid as a usage example but still not be fully migratable as-is.

In your example, the note is that the model is **not migratable as-is** because column types and lengths are not specified.

That is a very useful distinction for the docs:

- some examples are about runtime usage shape
- some examples are about complete schema intent
- those are not always the same thing

This should be called out clearly whenever model examples are introduced.

Model design guidance

A good default approach is:

- start with `RelmModel` when the standard metadata fields fit
- use attributes to keep mapping intent close to the model

- introduce `RelmModelClean` when you need a more customized shape
- treat relationship properties as part of the model's relational story, not just convenience fields

Where to go next

After this page, the best next steps are:

1. Review Contexts and DataSets
2. Review Options and Configuration

Those pages explain how models are used at runtime once they have been defined.

Contexts and DataSets

This page explains how CoreRelm organizes runtime access through contexts and data sets.

If models describe what your entities look like, contexts and datasets describe how you work with them at runtime.

The role of `RelmContext`

`RelmContext` is the main runtime object used to work with CoreRelm. It is where configuration, connection behavior, transaction behavior, model access, and lower-level execution helpers come together.

In a typical application, the context becomes the main entry point for:

- opening and using a configured runtime
- accessing model-oriented datasets
- beginning and controlling transactions
- coordinating work across multiple entities

The role of `RelmDataSet<T>`

A `RelmDataSet<T>` is the model-oriented access surface for a specific entity type. It is where common operations such as querying, creating, loading, and referencing related data are expressed.

From a design perspective:

- the model describes the entity
- the dataset is the access surface for that entity
- the context is where datasets and runtime behavior come together

Defining a context

Your updated Getting Started page describes the context as the main runtime object and shows an `ExampleContext` that exposes model-oriented data access members.

Example context shape

```
public class ExampleContext(RelmContextOptions contextOptions) : RelmContext(contextOptions)
{
    public IRelmDataSet<ExampleModel>? ExampleModels { get; set; }
    public IRelmDataSet<ExampleGroup>? ExampleGroups { get; set; }
}
```

This is a good mental model for how a context is typically defined:

- inherit from `RelmContext`
- accept `RelmContextOptions`
- expose one or more datasets for your mapped entities

What a context is responsible for

A CoreRelm context usually carries responsibility for four things:

1. Configuration

The context is created from `RelmContextOptions`, which determine how it behaves at runtime. That includes connection and initialization behavior.

2. Dataset access

The context is the home of datasets such as `ExampleModels` and `ExampleGroups`. This is how model-specific data access becomes discoverable and organized.

3. Transactions

The context is where explicit transaction control usually lives. Your Getting Started example shows a clear `BeginTransaction` / `CommitTransaction` / `RollbackTransaction` pattern.

4. Lower-level helpers

The context also provides access to lower-level operations such as identity helpers, dataset retrieval, and other direct runtime behaviors. Your Quick Start in 60 Seconds example uses helpers like `GetLastInsertId`, `GetIdFromInternalId`, and `GetDataSet<T>()`.

Context creation patterns

The updated Getting Started page shows a simple build pattern:

```
var relmContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

This is a very useful pattern because it keeps configuration and context creation readable and close together.

Datasets as the working surface

Once the context exists, datasets become one of the main working surfaces.

For example:

```
var exampleActiveModels = context
    .ExampleModels
    .Where(model => model.Active == true && model.Id > 5)
    .OrderByDescending(model => model.CreateDate)
    .Load();
```

This demonstrates the general idea:

- start from a context
- work through the relevant dataset
- express a query
- materialize the result shape you need

Lazy dataset access

Your updated Getting Started page also shows a manual dataset-access pattern:

```
if (context.ExampleModels == null)
    context.GetDataSet<ExampleModel>();
```

This is important because it connects directly to the eager versus quick context distinction. In a quick context, datasets may not be initialized up front, so retrieving them manually becomes part of the usage pattern.

Eager versus quick context behavior

Your updated Getting Started page elevates this distinction clearly, and it belongs in the core conceptual docs.

Eager context

An eager context initializes datasets up front and verifies tables before later operations need them. This favors readiness and can reduce later metadata work.

Quick context

A quick context disables automatic dataset initialization and automatic table verification. This reduces up-front work and allows metadata to load as needed. This choice is not just a minor configuration detail. It changes how the context behaves at runtime and how datasets are accessed in practice.

Scoped usage and transactions

Contexts are also the place where transaction boundaries become visible.

Example scoped transaction pattern

```
using var scopedContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();

scopedContext.BeginTransaction();

try
{
    // Perform database operations here

    scopedContext.CommitTransaction();
}
catch
{
    scopedContext.RollbackTransaction();
    throw;
}
```

This pattern is worth emphasizing because it reflects one of CoreRelm's central design preferences: transaction handling should be explicit and readable.

Design guidance

A good default way to think about contexts and datasets is:

- the context is the configured runtime shell
- datasets are the model-specific entry points
- transactions belong where the operation boundary is clear
- eager versus quick behavior should be a deliberate choice

Where to go next

After this page, the best next step is Options and Configuration.

That page explains how context behavior is configured and how the builder patterns shown here fit together.

Options and Configuration

This page explains how CoreRelm contexts are configured and how `RelmContextOptions` and `RelmContextOptionsBuilder` shape runtime behavior.

CoreRelm treats configuration as part of the readable runtime story. Rather than hiding important behavior behind distant configuration systems, the library makes options visible near the point of use.

Why options matter

Context options control how a CoreRelm context behaves. That includes:

- how the context connects
- whether the connection opens automatically
- whether a transaction opens automatically
- whether data sets initialize eagerly
- whether tables are verified automatically
- related provider/runtime settings

This means configuration is not just setup boilerplate. It directly affects the runtime shape of the library.

`RelmContextOptions`

`RelmContextOptions` is the object that represents runtime configuration for a context.

In practical terms, it is the packaged result of your configuration choices. Once built, it is used to construct the context that your application will work through.

`RelmContextOptionsBuilder`

`RelmContextOptionsBuilder` is the main entry point for creating those options. Your updated Getting Started page and README revisions both treat it as the standard configuration path.

Constructor styles shown in the QuickStart

Your updated Getting Started page calls out several builder constructor styles shown in the QuickStart project:

- starting from defaults
- copying from an existing options object
- initializing from an enum-backed connection reference
- initializing from a connection
- initializing from a connection and transaction
- initializing from a named connection string

- initializing from a raw connection string
- initializing from explicit server, database, user, and password values
- initializing from explicit server, port, database, user, and password values

This is a useful part of the API design because it allows applications to start with the most natural entry point for their environment.

Simple build pattern

A compact configuration pattern looks like this:

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

This is often the clearest starting point for a new application because it keeps the configuration shape minimal while still using the standard builder pathway.

Example maximal options shape

Your updated Getting Started page also includes a fuller options example:

```
var options = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoOpenConnection(true)
    .SetAutoOpenTransaction(false)
    .SetAllowUserVariables(false)
    .SetConvertZeroDateTime(false)
    .SetLockWaitTimeoutSeconds(0)
    .SetAutoInitializeDataSets(true)
    .SetAutoVerifyTables(true)
    .BuildOptions();
```

This is helpful because it shows that options are not only about the connection string. They also control runtime behavior such as connection opening, transaction behavior, and dataset/table initialization.

Eager versus quick configuration

One of the most important runtime choices is whether you want eager or quick behavior.

Eager configuration

An eager context initializes datasets up front and verifies tables up front.

```
var options = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoInitializeDataSets(true)
```

```
    .SetAutoVerifyTables(true)
    .BuildOptions();
```

This favors readiness and can reduce later metadata work.

Quick configuration

A quick context disables automatic dataset initialization and automatic table verification.

```
var quickOptions = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoInitializeDataSets(false)
    .SetAutoVerifyTables(false)
    .BuildOptions();
```

This reduces up-front work and allows metadata to load as needed.

This distinction should be treated as a first-class configuration choice because it affects both startup behavior and later runtime work.

Build patterns

Your docs now show two practical build styles.

Build options first, then create the context

```
var options = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoInitializeDataSets(true)
    .SetAutoVerifyTables(true)
    .BuildOptions();

using var context = new ExampleContext(options);
```

This can be useful when you want to separate configuration from context construction.

Build the context directly from the builder

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

This keeps common usage short and readable.

How to choose a configuration style

A good default strategy is:

- start with the simplest builder constructor that matches your environment
- override only the options you actually need
- make eager versus quick behavior a deliberate choice
- keep configuration readable and near the usage that depends on it

Your updated Getting Started page states this directly, and it is good advice for the broader docs as well.

Configuration and transaction behavior

Options can also influence transaction behavior. For example, your updated Getting Started page includes an example that enables auto-open transaction behavior:

```
using var relmContext = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoOpenTransaction(true)
    .Build<ExampleContext>();
```

That can be useful in some flows, but the broader documentation still emphasizes explicit transaction boundaries as the most readable default.

Configuration philosophy

The design preference behind the options system is straightforward:

- configuration should be understandable
- important runtime behavior should be visible
- developers should not have to guess how the context will behave

That fits the larger CoreRelm style of explicitness over magic.

Where to go next

After this page, the next best step is to move into the Working with Data section.

That is where the configured context, data sets, and models start to come together in CRUD, querying, relationships, DTOs, loaders, and bulk operations.

CRUD Operations

This page explains the basic create, read, update, and delete workflow shape in CoreRelm.

CoreRelm is designed to make common relational work more structured and readable while still keeping the database visible in the way you work. In practice, that means CRUD operations are usually expressed through a configured context and its data sets.

The basic CRUD flow

A typical CRUD workflow in CoreRelm looks like this:

1. Build or create a context
2. Access the relevant data set
3. Create, load, or query a model
4. Change the model or write a new one
5. Use explicit transactions when the operation spans multiple writes

This is intentionally direct. CoreRelm prefers explicitness over hidden behavior.

Create

A common create flow starts from the context and a data set.

Example shape

```
var person = context.People.New();
person.FirstName = "Ada";
person.LastName = "Lovelace";
person.WriteToDatabase(context);
```

This pattern is useful because it keeps the operation readable:

- start from the configured runtime
- create a new model instance through the relevant data set
- populate values
- write the model using the context

Depending on the surrounding workflow, you may also use model-level save patterns in other examples. The important concept is that the create flow remains visible and easy to follow.

Read

CoreRelm supports both direct lookup and broader query-style retrieval.

Find a single model

```
var found = context.People.Find(person.InternalId);
```

This is the narrow "get me this one model" shape.

Query for multiple models

```
var results = context.People
    .Where(x => x.LastName == "Lovelace")
    .OrderBy(x => x.FirstName)
    .Load();
```

This is the broader "filter, shape, and materialize" flow.

Update

A common update shape is:

1. Load or find a model
2. Change one or more properties
3. Persist the updated model

Example shape

```
var found = context.People.Find(person.InternalId);

if (found != null)
{
    found.LastName = "Byron";
    found.WriteToDatabase(context);
}
```

In CoreRelm, the update step is usually easy to reason about because the model is explicit and the persistence step is visible.

Delete

Delete examples should follow the same philosophy as the rest of the library:

- make the boundary clear
- make the target clear
- make the persistence step clear

Example shape

```
var found = context.People.Find(person.InternalId);

if (found != null)
{
    found.DeleteFromDatabase(context);
}
```

Even if the exact helper methods vary by feature area, the documentation should keep the mental model consistent: load the target, then perform an explicit removal step.

Using data sets as the primary surface

For everyday CRUD work, the most common entry point is the model-specific data set exposed by the context.

That makes the code easier to navigate because the developer can usually answer:

- Which model am I working with?
- Which context owns it?
- Which data set is the access surface?

Save and write patterns

Across the documentation and examples, you may see slightly different persistence verbs depending on the example focus:

- `WriteToDatabase(...)`
- model-level save patterns
- data set batch-add flows followed by a save step

Those are all part of the same broader idea: CoreRelm keeps persistence visible in code instead of hiding it behind large amounts of implicit state tracking.

CRUD and transactions

For a single isolated write, a lightweight flow may be enough.

For multiple related writes, a better default pattern is:

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();
```

```
context.BeginTransaction();

try
{
    // create, update, or delete multiple related models here

    context.CommitTransaction();
}
catch
{
    context.RollbackTransaction();
    throw;
}
```

CoreRelm is most readable when transaction boundaries are explicit.

Design guidance

A good default CRUD mindset in CoreRelm is:

- begin from the context
- use the relevant data set
- keep model state changes readable
- make persistence steps explicit
- introduce explicit transactions when operations become multi-step

Where to go next

After this page, the best next steps are:

1. Querying and Result Shapes
2. Relationships and Foreign Keys

Those pages explain how CRUD grows into richer querying and relationship-oriented workflows.

Querying and Result Shapes

This page explains how CoreRelm expresses query workflows and how different result shapes fit into the overall data-access model.

CoreRelm uses a model- and dataset-centered query style. The goal is to keep queries readable, explicit, and close to the model being worked with.

Querying starts from a data set

A common query flow begins with the context and then moves into the model-specific data set.

Example shape

```
var exampleActiveModels = context
    .ExampleModels
    .Where(model => model.Active == true && model.Id > 5)
    .OrderByDescending(model => model.CreateDate)
    .Load();
```

This illustrates the general pattern:

- start from the configured context
- use the appropriate data set
- express the query conditions
- materialize the result

Common query steps

The QuickStart and broader documentation surface point to several common query-building ideas:

- filtering
- ordering
- grouping
- limiting
- offsets
- distinct selection patterns

Filtering

Filtering is usually expressed through `Where(...)`.

```
var filtered = context.ExampleModels
    .Where(model => model.Active == true && model.Id > 5)
```

```
    .Load();
```

Ordering

Ordering is usually expressed through `OrderBy(...)` or `OrderByDescending(...)`.

```
var ordered = context.ExampleModels
    .OrderByDescending(model => model.CreateDate)
    .Load();
```

Grouping, limits, and offsets

The QuickStart feature inventory indicates support for grouping, limits, and offsets in the broader data set workflow.

The purpose of these operations is to keep query intent close to the data-access call chain rather than pushing every query shape into a completely separate layer.

Result shapes

One of the important themes in the QuickStart project is that CoreRelm demonstrates multiple data result shapes, not only model-list loading.

The current docs and example inventory call out support for:

- data row access patterns
- data table access patterns
- data object access patterns
- data list access patterns

That is useful because not every query needs to materialize full model instances.

Model-oriented result shape

The most familiar result shape is loading one or more mapped models.

```
var models = context.ExampleModels
    .Where(model => model.Active == true)
    .Load();
```

This is usually the right choice when:

- you want mapped entity instances
- you want to keep working in a model-oriented workflow

- you want to continue into relationships or model-based logic

Object-oriented result shape

Sometimes you want a lightweight object-oriented result that is not centered on a mapped model instance.

Use this shape when:

- you need a projected or non-model object
- you want a lighter result
- the query does not naturally map to a full entity workflow

List-oriented result shape

A list result shape is useful when the main goal is to gather repeated values or repeated projected items.

This can be useful for:

- simple reporting flows
- quick data access helpers
- workflows where full model mapping would be unnecessary

Row-oriented result shape

A row-oriented result shape is useful when you want direct access to a single raw row structure without wrapping the result in a mapped model.

This is often a practical choice when:

- the result is ad hoc
- you need a database-shaped response
- you are closer to low-level database work

Table-oriented result shape

A table-oriented result shape is useful for broader result sets where a tabular structure is more natural than a mapped-model workflow.

This can be a good fit for:

- reporting
- export-like workflows
- internal admin or diagnostics scenarios

Querying philosophy

The main design idea is not that every workflow must become a model list.

Instead, CoreRelm tries to support several levels of access:

- model-oriented queries through data sets
- object/list/row/table-oriented result shapes when a different shape is a better fit
- lower-level database work helpers when you need more direct control

Choosing a result shape

A good default decision process is:

- use mapped models when you are working with real entities
- use projected or lightweight shapes when the result is more task-specific
- use row/table-oriented access when the query is closer to raw relational work
- keep the result shape aligned to the job, not to habit

Design guidance

CoreRelm is easiest to reason about when the query shape matches the usage shape:

- entity work should look like entity work
- reporting work should look like reporting work
- low-level data work should stay visibly low-level

That consistency helps keep the documentation and the code aligned.

Where to go next

After this page, the best next steps are:

1. Relationships and Foreign Keys
2. DTOs and Projection Patterns

Those pages explain how query results connect to related data and alternative shapes.

Relationships and Foreign Keys

This page explains how CoreRelm models describe relationships and how related data is loaded through foreign-key workflows.

CoreRelm keeps relationship intent close to the model by using attributes and readable navigation properties.

Relationship mapping at the model level

A common relationship shape in CoreRelm starts with a model property and a `RelmForeignKey` attribute.

Example shape

```
[RelmTable("example_models")]
public class ExampleModel : RelmModel
{
    [RelmColumn]
    public string? GroupInternalId { get; set; }

    [RelmForeignKey(foreignKey: nameof(ExampleGroup.InternalId),
localKey: nameof(GroupInternalId))]
    public virtual ExampleGroup Group { get; set; }
}

[RelmTable("example_groups")]
public class ExampleGroup : RelmModel
{
    [RelmColumn]
    public string? Name { get; set; }

    [RelmForeignKey(foreignKey: nameof(ExampleModel.GroupInternalId),
localKey: nameof(InternalId))]
    public virtual ICollection<ExampleModel> Models { get; set; }
}
```

This demonstrates a few important ideas:

- the local key is visible on the model
- the foreign key target is visible on the model
- relationship direction is easy to read
- navigation intent stays close to the entity definition

Why foreign-key metadata matters

Foreign-key metadata helps CoreRelm understand how one model relates to another. That matters for:

- readable navigation properties
- relationship loading
- model-oriented queries
- schema and migrations workflows

Single-reference navigation

A single-reference navigation points from one entity to a related parent or peer.

Example

```
[RelmForeignKey(foreignKey: nameof(ExampleGroup.InternalId),
localKey: nameof(GroupInternalId))]
public virtual ExampleGroup Group { get; set; }
```

This is a good fit when one model belongs to or references another model.

Collection navigation

A collection navigation points from one entity to a related set of other entities.

Example

```
[RelmForeignKey(foreignKey: nameof(ExampleModel.GroupInternalId),
localKey: nameof(InternalId))]
public virtual ICollection<ExampleModel> Models { get; set; }
```

This is a good fit when one model owns or groups many related models.

Loading related data

The QuickStart inventory shows dedicated foreign-key loading examples and data set reference behavior. That indicates CoreRelm expects related-data loading to be a first-class part of the workflow.

At a high level, there are two important concepts here:

- mapping the relationship clearly
- loading or referencing the related data when needed

Relationship loading patterns

A relationship workflow usually looks like one of these:

Load the main model, then load references

```
var models = context.ExampleModels
    .Where(model => model.Active == true)
    .Load();

// relationship loading or reference-loading step here
```

Query through a data set and use reference-oriented helpers

CoreRelm also supports relationship-oriented access patterns through data sets and reference-style workflows.

This keeps relationship logic attached to the model and dataset surfaces rather than forcing every relationship query into raw SQL.

Non-default and advanced foreign-key shapes

The broader codebase inventory indicates foreign-key navigation and compound-key testing in the repository.

That suggests the relationship system is meant to support more than only the simplest one-column default case.

This is worth documenting clearly over time:

- default key relationships
- non-default local and foreign keys
- compound-key scenarios
- bidirectional relationship navigation patterns

Runtime versus schema examples

As with other CoreRelm docs pages, it is important to distinguish between:

- examples that show usage shape
- examples that fully define migratable schema intent

A relationship example may be excellent for teaching runtime navigation while still not being a full migrations-ready schema definition.

Design guidance

A good default approach to relationships in CoreRelm is:

- keep key fields visible on the model
- use `RelmForeignKey` to make the relationship readable
- treat navigation properties as part of the relational story
- keep relationship loading explicit enough that readers can follow it

Where to go next

After this page, the best next steps are:

1. DTOs and Projection Patterns
2. Data Loaders and Field Loaders

Those pages explain alternative ways to shape and load related data.

DTOs and Projection Patterns

This page explains how DTO-style workflows fit into CoreRelm and when a DTO is a better fit than a mapped model.

CoreRelm supports model-oriented work, but not every query or response should become a full entity model. DTOs are useful when the shape you need is task-specific rather than entity-specific.

Why DTOs exist

A mapped model is usually the right fit when:

- you are working with a real entity
- you want entity-oriented persistence
- you want the model to carry relational metadata

A DTO is often the better fit when:

- the shape is for transfer or presentation
- the result combines values for a specific task
- you want a lighter object than a full mapped entity
- persistence is not the main concern

DTOs in the QuickStart surface

The QuickStart project includes dedicated DTO examples. That is a strong signal that DTO usage is a real public feature area rather than an afterthought.

That also fits the broader documentation philosophy: the library should support several useful data shapes instead of forcing everything into one pattern.

DTOs versus models

A practical way to think about the difference is:

Use a model when:

- the object represents a database entity
- the object needs relational metadata
- the object participates directly in model-oriented workflows

Use a DTO when:

- the shape is tailored to a specific query or response
- the object is primarily for transfer or display

- you do not want to carry full entity semantics into the result

Projection patterns

Projection is the broader idea of shaping results into the object you actually need.

A projection-oriented workflow is useful when:

- a full model would include unnecessary data
- the consuming code only needs a smaller shape
- the query result crosses multiple concepts but is not itself a true entity

Example DTO shape

```
public class ExampleSummaryDto
{
    public string? Name { get; set; }
    public string? GroupName { get; set; }
    public int Value { get; set; }
}
```

This kind of shape is useful for:

- summaries
- reporting
- API responses
- UI-facing data objects

DTO workflow mindset

When you document or design DTO flows in CoreRelm, keep the following mental model:

- models are for mapped relational entities
- DTOs are for shaped results
- DTOs should make the result easier to consume
- DTOs should not be forced to act like entities if they are not entities

DTOs and query result shapes

DTO usage connects naturally to the broader result-shape story in CoreRelm.

When a query does not need a full model result, a DTO or projected object may be a better choice than:

- loading a full entity
- manually unpacking row/table results later

- carrying unnecessary relational fields through the application

Design guidance

A good default approach is:

- start with a model when the workflow is truly entity-based
- switch to a DTO when the result is clearly task-specific
- keep DTOs small and purposeful
- name DTOs by what they are for, not just by where they came from

Where to go next

After this page, the best next steps are:

1. Data Loaders and Field Loaders
2. Bulk Operations

Those pages explain additional ways CoreRelm supports specialized data-access workflows.

Data Loaders and Field Loaders

This page explains how data loaders and field loaders fit into the CoreRelm model.

CoreRelm includes loader-oriented feature areas in the QuickStart and codebase. These patterns are useful when the default data-loading flow is not enough and you want a more specialized loading strategy.

Why loaders exist

A standard dataset query is often enough for common entity work.

Loaders become useful when:

- a field needs custom load behavior
- related or derived values need special handling
- you want a reusable loading pattern
- the loading logic should be made explicit rather than hidden in scattered ad hoc code

Loader-oriented feature areas

The QuickStart inventory includes:

- data loader examples
- field loader files
- dedicated loading examples tied to data access workflows

That indicates loaders are meant to be a supported concept, not a one-off internal trick.

Data loaders

A data loader is best understood as a reusable strategy for loading or shaping data beyond the simplest default path.

Use a data loader when:

- the same loading behavior should be reused
- the logic is important enough to name and document
- you want a clearer boundary around data-shaping logic

Field loaders

A field loader is a narrower version of the same idea. It focuses on a particular field or field-like behavior.

A field loader can be useful when:

- one field needs special lookup logic
- a field is derived from a more complex loading path
- a property should be filled by a clearly named reusable mechanism

Loader mindset

The important design idea is this:

- keep common loading logic reusable
- keep special loading logic discoverable
- avoid burying repeated data-shaping behavior in many unrelated call sites

That makes the data-access layer easier to understand over time.

Loaders versus DTOs versus relationships

These three ideas are related, but they solve different problems:

- **relationships** describe how entities connect
- **DTOs** shape results for a task
- **loaders** define reusable loading behavior when the default path is not enough

Keeping those responsibilities distinct will make the documentation clearer.

Example loader-style mental model

A loader-oriented workflow often looks like this:

1. define the loader concept
2. attach or reference it where appropriate
3. use it as part of a query or load operation
4. keep the loading behavior reusable and understandable

Design guidance

A good default approach is:

- do not reach for loaders first when a simple query is enough
- introduce loaders when the behavior is repeated or important
- keep loaders focused and clearly named
- document what the loader is responsible for and what it is not responsible for

Where to go next

After this page, the best next steps are:

1. Bulk Operations
2. Database Work Helpers

Those pages cover performance-oriented and lower-level data workflows.

Bulk Operations

This page explains how bulk operations fit into CoreRelm and when they are a better fit than ordinary model-by-model write flows.

CoreRelm includes dedicated bulk table writing examples in the QuickStart project. That signals a deliberate distinction between standard entity-oriented persistence and higher-throughput write scenarios.

Why bulk operations exist

A standard write flow is often the clearest option when:

- the number of records is small
- readability matters more than throughput
- entity-level logic is important

Bulk operations become useful when:

- many rows need to be written
- throughput matters
- the overhead of one-model-at-a-time persistence would be too high
- the workflow is more data-movement-oriented than entity-lifecycle-oriented

Bulk table writing

The QuickStart inventory includes:

- `BulkTableWriterExamples`
- `WriteToDatabaseExamples`

That suggests the docs should treat bulk writing as its own feature area, not just a footnote under CRUD.

Bulk operations versus standard writes

A practical way to think about the difference is:

Standard writes

Use standard write flows when:

- the operation is entity-centric
- per-model logic matters
- clarity is more important than maximum throughput

Bulk writes

Use bulk flows when:

- the operation is row-volume-centric
- the goal is efficient ingestion or movement
- you are working with larger batches

Bulk operation tradeoffs

Bulk operations usually come with tradeoffs that should be called out clearly in the docs:

- they may be less model-centric
- they may bypass some of the feel of ordinary entity workflows
- they are often optimized for throughput rather than per-entity expressiveness

That does not make them worse. It just makes them better suited to a different kind of job.

Bulk operations and transactions

Bulk workflows should still be documented with explicit reliability boundaries in mind.

A good default approach is:

- decide whether the whole batch should succeed or fail together
- use explicit transaction boundaries when appropriate
- keep rollback behavior understandable

Design guidance

A good default approach is:

- start with standard write flows when the scale is modest
- move to bulk writing when volume makes it worth it
- document the operational goal clearly
- treat bulk workflows as performance-oriented tools, not as the default for every write

Where to go next

After this page, the best next step is Database Work Helpers.

That page covers the lower-level helper side of the data-access story.

Database Work Helpers

This page explains where lower-level database work helpers fit into CoreRelm.

CoreRelm is not trying to hide the database completely. One of its design goals is to give you structure while still keeping relational and connection-level details understandable. Database work helpers are part of that philosophy.

Why database work helpers exist

A model- and dataset-centered workflow is often the best default.

But some situations call for a more direct shape, such as:

- administrative operations
- reporting-oriented access
- raw or specialized data retrieval
- tasks that do not naturally fit a full model workflow
- utility operations that are easier to express closer to the database

Database work in the QuickStart surface

The QuickStart inventory includes a dedicated `DatabaseWorkExamples` file, along with separate examples for data rows, data tables, data objects, and data lists.

That suggests CoreRelm intentionally supports a layered data-access model:

- entity-oriented access when you want it
- lower-level access when you need it

Lower-level helpers and identity utilities

The current docs and examples also show direct helper-style operations such as:

- `GetLastInsertId()`
- `GetIdFromInternalId(...)`
- `GetDataSet<T>()`
- `RelmHelper.GetDalTable<T>()`

These are useful because they support practical relational workflows without forcing the developer to abandon the broader CoreRelm model.

Example helper shape

```
var lastInsertId = context.GetLastInsertId();

var tableName = RelmHelper.GetDalTable<ExampleModel>();
var internalId = "some-guid-value";
var idFromInternalId = context.GetIdFromInternalId(tableName, internalId);
```

This is a good example of CoreRelm's design style:

- still structured
- still model-aware
- still explicit about the database

When to go lower level

A good time to use lower-level database helpers is when:

- the job is not naturally entity-oriented
- you need a data shape other than full models
- the query is closer to raw relational logic
- the helper makes the intent clearer than forcing a model path would

Lower-level does not mean abandoning structure

One of the strengths of CoreRelm is that even lower-level work can still remain part of a readable, structured system.

That means lower-level access should still aim to be:

- discoverable
- intentional
- well-bounded
- understandable in the surrounding context

Design guidance

A good default approach is:

- start with models and datasets when the workflow is entity-based
- move to lower-level database helpers when the job clearly calls for it
- do not force everything into one abstraction level
- keep helper usage explicit and easy to trace

Where to go next

After this page, the next best step is to move into the Transactions, Migrations, and Advanced Topics section.

That is where reliability, schema workflows, and more advanced runtime patterns come together.

Transactions and Unit of Work

This page explains how transaction handling fits into CoreRelm and why explicit transaction boundaries are one of the library's core design preferences.

CoreRelm favors making database boundaries visible. When an operation spans multiple related writes, the transaction should usually be easy to see in the code.

Transaction philosophy

CoreRelm is most readable when transaction handling is explicit.

A good default pattern is:

- begin a transaction when an operation spans multiple writes
- commit only after all related operations succeed
- roll back on failure
- keep error paths simple and predictable

This matches the broader design style of the library: explicitness over hidden behavior.

Standard explicit transaction pattern

A common transaction workflow looks like this:

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();

context.BeginTransaction();

try
{
    // Perform multiple related operations here

    context.CommitTransaction();
}
catch
{
    context.RollbackTransaction();
    throw;
}
```

This is often the clearest and safest default for multi-step write work.

Why explicit transactions are recommended

Explicit transactions make several things easier to reason about:

- where a unit of work starts
- where a unit of work ends
- what should succeed together
- what should roll back together
- where error handling belongs

That is especially useful in an ORM that is intentionally trying to stay close to relational reality.

Auto-open transaction behavior

CoreRelm also supports configuration that can automatically open a transaction.

Example shape

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .SetAutoOpenTransaction(true)
    .Build<ExampleContext>();

try
{
    // database work here
}
catch
{
    context.RollbackTransaction();
    throw;
}
```

This can be useful in some flows, but the broader documentation still treats explicit transaction boundaries as the most readable default.

Unit of work mindset

A useful way to think about transaction handling in CoreRelm is in terms of a unit of work.

A unit of work is the set of related changes that should succeed or fail together.

That means transaction design should be driven by business and data boundaries, not just by convenience.

Contexts and transactions

The context is where transaction behavior usually lives.

That matters because the context is already the place where:

- configuration comes together
- dataset access is exposed
- connection behavior is managed
- lower-level helpers are available

Keeping transactions there makes the workflow easier to follow.

Error handling and rollback

A good default rollback mindset is:

- roll back as soon as the multi-step operation is known to have failed
- keep the rollback path simple
- log application and database diagnostics where useful
- rethrow or wrap exceptions at the appropriate boundary

Design guidance

A good default approach is:

- use explicit transactions for multi-step writes
- keep the transaction boundary close to the operations it protects
- prefer clarity over cleverness
- treat auto-open transaction behavior as an optional tool, not as the universal default

Where to go next

After this page, the best next steps are:

1. Migrations Overview
2. Generating Migrations

Those pages explain how CoreRelm handles schema-oriented workflows in addition to runtime operations.

Migrations Overview

This page explains what migrations mean in CoreRelm and why migrations should be treated as a first-class part of the public feature set.

CoreRelm includes migrations-related APIs and tooling as part of its broader ORM story. Migrations are not just an implementation detail; they are part of how the library approaches schema evolution and relational consistency.

Why migrations exist

A runtime ORM story is only part of the picture.

Applications also need a way to think about:

- schema changes over time
- desired model shape versus current database shape
- reviewable change history
- safe rollout of structural changes

CoreRelm's migrations tooling exists to support those needs.

Migrations as part of the public surface

The current project materials make it clear that migrations are a major feature area.

That includes:

- public migrations-related documentation and examples
- migrations tests
- model set parsing and resolver behavior
- migration tooling and validation workflows
- schema-oriented helper and rendering layers

This is why migrations deserve their own section in the guide rather than just API reference pages.

High-level migrations mental model

A useful way to think about migrations in CoreRelm is:

1. define or resolve the desired schema shape
2. compare that desired shape to the current database state
3. generate or plan the changes needed
4. review the resulting migration artifacts
5. apply the migration safely

That is the broad workflow the rest of this section builds on.

Model sets and desired schema

One of the central ideas in the CoreRelm migrations surface is the model set.

A model set represents the schema-relevant view of the models that should define the desired database shape.

That matters because migrations need a stable source of truth for what the database is expected to look like.

Migrations are more than script generation

A good migrations system is not only about producing SQL.

It is also about:

- naming and organizing changes
- validating assumptions
- detecting drift
- making rollout safer
- keeping schema history understandable over time

The CoreRelm migrations surface should be documented with that broader perspective in mind.

Reviewability matters

CoreRelm's documentation should treat generated migrations as reviewable artifacts.

A good default stance is:

- prefer explicit, readable migration outputs
- review generated changes before applying them
- use tests and staging environments for confidence
- treat migration files and generated SQL as part of the deployment story

Runtime examples versus migration-ready definitions

As elsewhere in the docs, it is important to distinguish between:

- examples that show runtime usage shape
- examples that fully define schema intent

Not every runtime model example is necessarily enough to produce a complete migration on its own.

Design guidance

A good default approach is:

- treat migrations as deliberate, reviewable schema changes
- keep naming and organization consistent
- understand the difference between desired schema and current database state
- use validation and status checks before rollout where possible

Where to go next

After this page, the best next steps are:

1. Generating Migrations
2. Applying Migrations
3. Drift Detection and Validation

Generating Migrations

This page explains the role of migration generation in CoreRelm and how to think about it safely.

Generating a migration is the point where the desired schema model becomes a concrete change plan or artifact that can be reviewed and eventually applied.

The goal of generation

Migration generation should answer a simple question:

> Given the desired schema and the current database state, what change artifact should be produced?

That artifact should be understandable enough to review and reliable enough to use as part of a controlled schema workflow.

High-level generation flow

A good mental model for generation is:

1. resolve the desired schema input
2. compare it to the current state
3. build a migration plan
4. render the resulting artifact
5. review the output before applying it

This is one reason the documentation should separate generation from application. They are related, but they are not the same responsibility.

Example usage shape

```
var migrationTool = new RelmMigrationTooling();
var result = migrationTool.GenerateMigration(/* model set / options */);

if (!result.Success)
{
    // inspect errors
}
```

This is intentionally documentation-style and focuses on call shape rather than complete runnable setup.

Naming and organization

Generated migrations are easier to maintain over time when naming is consistent.

Good documentation guidance should include:

- use clear, descriptive names
- keep ordering understandable
- avoid ambiguous migration purpose
- organize files in a way that makes history readable

Review before apply

Generation should usually be followed by review.

That review may include:

- reading the generated SQL or migration script
- checking intended object changes
- confirming table, key, and constraint behavior
- verifying that the migration matches the desired change, not just that it exists

Generation inputs

The broader CoreRelm migrations surface suggests that generation depends on more than one thing, including:

- model sets
- current schema state
- migration planning logic
- rendering logic

That is why generation should be documented as a workflow step, not just as a single method call.

Generation and tests

Migration generation benefits from tests because schema workflows are easy to break in subtle ways.

The repository materials indicate test coverage around migrations planning and parsing, which is a strong signal that generation should be treated carefully.

Design guidance

A good default approach is:

- generate with clear intent
- review the artifact before applying it
- keep naming consistent
- treat migration outputs as part of the versioned schema story

Where to go next

After this page, the best next steps are:

1. Applying Migrations
2. Drift Detection and Validation

Applying Migrations

This page explains how to think about applying migrations in CoreRelm and why application should be treated as a deliberate deployment step.

Applying a migration is where schema intent becomes an actual database change. Because of that, apply workflows should be explicit, reviewable, and operationally careful.

The goal of application

The purpose of applying a migration is to move the database from its current state toward the desired state in a controlled way.

That means application is not just a technical step. It is also an operational step.

Example usage shape

```
var migrationTool = new RelmMigrationTooling();
var applyResult = migrationTool.ApplyMigrations(/* connection / options */);
```

As with the rest of the QuickStart-style documentation, this example is intended to show the call shape rather than a full runnable workflow.

Why apply should be separated from generate

Generation and application should usually be documented separately because they answer different questions:

- generation asks what change artifact should exist
- application asks when and how the change should be executed

Keeping those responsibilities separate improves clarity and reduces accidental rollout risk.

Safe rollout mindset

A good default apply mindset is:

- review the migration before applying it
- understand which environment you are targeting
- prefer staged rollout where appropriate
- verify success after application
- have a rollback or recovery plan where possible

Apply results and diagnostics

Application workflows should be documented with diagnostics in mind.

Developers will want to know:

- did the migration apply successfully?
- what failed if it did not?
- what object or step caused the failure?
- what state is the database in afterward?

That is one reason apply-result types and related diagnostics deserve documentation attention.

Applying in different environments

Apply guidance should distinguish between environments such as:

- local development
- integration or staging
- production

A behavior that is fine during local iteration may not be appropriate for production rollout.

Operational guidance

Good operational guidance for applying migrations includes:

- avoid applying unreviewed changes directly to production
- use staging verification when possible
- keep backups and recovery strategy in mind
- log migration execution details
- verify post-apply state

Design guidance

A good default approach is:

- generate first
- review second
- apply deliberately
- verify afterward
- keep the migration history understandable

Where to go next

After this page, the next best step is [Drift Detection and Validation](#).

That page covers how CoreRelm helps compare intended schema state with actual database state.

Drift Detection and Validation

This page explains how validation and drift detection fit into the CoreRelm migrations story.

Schema work is not only about creating migrations. It is also about making sure the database and the intended model remain aligned over time.

What drift means

Schema drift is the gap between:

- the desired schema state
- the actual database state

Drift can happen for many reasons, including:

- manual database changes
- partially applied changes
- schema differences between environments
- model evolution without matching migration discipline

Why validation matters

Validation helps answer questions such as:

- is the schema still aligned with the intended model?
- are required migration steps missing?
- does the current state match what the tooling expects?
- is it safe to proceed with the next schema step?

That makes validation one of the most practical safety features in a migrations system.

Drift detection as an ongoing workflow

Drift detection should not be treated as a one-time step.

It is useful:

- before generating migrations
- before applying migrations
- after applying migrations
- when comparing environments
- when investigating unexpected schema behavior

Diagnostic mindset

When documenting validation and drift detection, the guide should help readers answer:

- what is out of sync?
- how severe is the mismatch?
- is it a naming issue, type issue, key issue, or constraint issue?
- what should be done next?

That will make this section more useful than a simple feature checklist.

Validation inputs

The broader CoreRelm migrations materials suggest validation depends on several components working together, including:

- model-set parsing or resolution
- desired schema construction
- current-schema inspection
- migration planning and comparison logic

That is why validation deserves its own explanation instead of being hidden inside other migration pages.

Common validation situations

The most useful cases to document are:

- expected clean alignment
- missing migration or unapplied changes
- manual database drift
- object mismatch such as columns, keys, or constraints
- environment mismatch between staging and production

Design guidance

A good default approach is:

- validate before applying important changes
- treat drift as a real signal, not as a nuisance
- investigate the source of mismatch instead of papering over it
- document the next action clearly whenever validation fails

Where to go next

After this page, the best next steps are:

1. Schema Metadata Features
2. Dependency Injection and Integration

Schema Metadata Features

This page explains the broader schema-oriented metadata features exposed by CoreRelm beyond basic table and column mapping.

CoreRelm's public surface includes schema-related concepts that go beyond simple entity mapping. These features are important both for runtime understanding and for migrations/schema workflows.

Why schema metadata matters

Schema metadata is where relational intent becomes more precise.

It helps describe things such as:

- identity and keys
- indexes
- uniqueness constraints
- foreign keys
- triggers
- functions
- procedures
- database-level features

This is one of the reasons CoreRelm can support both runtime ORM workflows and schema-oriented workflows in the same ecosystem.

More than tables and columns

A new reader may first encounter CoreRelm through `RelmTable` and `RelmColumn`, but the schema story is broader than that.

The broader metadata surface helps the library express:

- how an entity is identified
- how it relates to other entities
- how constraints should behave
- what additional database objects participate in the schema

Keys and identity

Keys and identity metadata are central because they affect:

- entity lookup
- update and delete targeting
- relationship behavior

- schema generation and migration planning

Indexes and uniqueness

Indexes and uniqueness-related metadata matter for:

- performance
- data integrity
- query behavior
- schema planning

These should be documented not only as attributes or types, but also in terms of why a developer would choose them.

Foreign keys

Foreign-key metadata connects this page back to the relationships section.

The schema perspective adds another important layer:

- the relationship is not only a navigation concern
- it is also a database constraint and schema concern

Triggers, functions, and procedures

The broader project inventory shows dedicated attribute and API surface coverage for triggers, functions, procedures, and related concepts.

That means the documentation should treat them as real schema features, not as obscure edge cases.

These features are useful when:

- database behavior needs to include server-side logic
- the schema story includes more than plain tables
- migrations and schema tooling must understand more than simple entity mapping

Database-level metadata

The broader schema story can also include database-level attributes or provider-specific schema features.

Because CoreRelm is currently focused on MySQL 8+, the docs should explain these features with that provider context in mind.

Design guidance

A good default approach is:

- document schema metadata as part of relational intent
- explain both runtime impact and schema impact
- keep examples clear about whether they are usage examples or full schema examples
- connect schema metadata docs to the migrations section whenever relevant

Where to go next

After this page, the next best step is Dependency Injection and Integration.

Dependency Injection and Integration

This page explains where dependency injection and broader application integration fit into CoreRelm.

CoreRelm is designed around explicit context construction, but that does not mean it cannot fit into larger application architectures. Integration guidance helps teams use CoreRelm cleanly inside services, web applications, background processors, and other hosted environments.

Why integration guidance matters

Even a clear library becomes harder to use if the surrounding application shape is not well understood.

Integration documentation helps answer:

- where should contexts be created?
- how should configuration be supplied?
- where should transaction boundaries live?
- how should CoreRelm fit into a service-based application?

Explicitness still matters

A useful principle for integration is:

> dependency injection should help manage structure, not hide important database behavior

That means DI should support CoreRelm's design style rather than pushing it toward invisible behavior.

Service boundaries and context usage

A good default approach is to keep context usage close to the application service or operation that owns the unit of work.

That usually means:

- create or resolve the context at a clear application boundary
- keep transaction handling near the business operation it protects
- avoid spreading one implicit unit of work across too many unrelated layers

Configuration and environment wiring

Integration also includes deciding how runtime configuration enters the system.

That may include:

- named connection strings

- environment-specific configuration
- startup wiring
- app-level service registration

Those choices should preserve readability and make the runtime behavior easier, not harder, to understand.

Dependency injection support

The broader CoreRelm public surface includes service-collection integration support, which suggests DI is a supported advanced topic rather than only a user invention.

That means this page should eventually document:

- the available DI extension surface
- recommended registration patterns
- where to resolve contexts
- how to avoid lifetime confusion

Integration patterns by app type

Over time, this page should distinguish between common application shapes such as:

- console or utility applications
- web applications
- background processing services
- integration or migration runners

Each of those may prefer slightly different context and transaction patterns.

Design guidance

A good default approach is:

- let DI manage construction and configuration when it helps
- keep database behavior explicit
- keep unit-of-work boundaries easy to see
- avoid hiding CoreRelm behind layers that make troubleshooting harder

Where to go next

After this page, the next best step is to move into the Troubleshooting, Reference, and Project Guidance section.

That is where error handling, API reference guidance, QuickStart mapping, architecture notes, and contribution guidance come together.

Error Handling and Diagnostics

This page explains how to think about error handling and diagnostics in CoreRelm.

CoreRelm is most effective when database behavior stays visible. That includes failure paths. A good error-handling strategy should make it easy to see what failed, where it failed, and what should happen next.

Core error-handling mindset

A good default approach in CoreRelm is:

- make transaction boundaries explicit
- roll back multi-step operations when they fail
- keep error paths simple
- log useful application and database context
- avoid hiding failures behind overly clever abstractions

This fits the broader design preference of explicitness over magic.

Error handling and transactions

The most important error-handling pattern in CoreRelm is usually the transaction boundary.

Standard pattern

```
using var context = new RelmContextOptionsBuilder("name=ExampleConnectionString")
    .Build<ExampleContext>();

context.BeginTransaction();

try
{
    // multi-step database work here

    context.CommitTransaction();
}
catch
{
    context.RollbackTransaction();
    throw;
}
```

This pattern is valuable because the success path and failure path are both easy to follow.

Why rollback matters

Rollback matters when multiple related operations should succeed or fail together.

Without a clear rollback path, it becomes harder to answer:

- which operations were completed
- which operations failed
- whether the database is in a partial state
- what the recovery action should be

Diagnostics and inspection

The broader CoreRelm docs direction already identifies diagnostics members such as:

- `HasError`
- `LastExecutionError`
- `LastExecutionException`

These kinds of diagnostics are useful because they let the developer inspect library- or database-related failure information without guessing blindly.

A practical diagnostic workflow

A useful diagnostic workflow is:

1. identify whether the operation failed
2. inspect any CoreRelm-exposed diagnostic state
3. inspect the application exception path
4. confirm whether a rollback occurred or should occur
5. log enough information to reproduce or investigate later

Migration-related diagnostics

Diagnostics matter for migrations too, not just runtime CRUD work.

Migration diagnostics should help answer questions such as:

- did generation fail?
- did validation detect drift?
- did application fail on a particular object or step?
- what environment was affected?

That is why troubleshooting and migrations should remain closely connected in the docs.

Common failure categories

The most useful categories to cover over time are:

- configuration problems
- connection or provider issues
- dataset initialization confusion
- relationship or key mismatches
- migration validation or apply failures
- unexpected result-shape or loading behavior

Design guidance

A good default approach is:

- make write boundaries explicit
- inspect diagnostics close to the failure
- do not swallow exceptions silently
- keep error handling boring, predictable, and readable

Where to go next

After this page, the best next step is FAQ.

That page collects common problems and answers in one place.

FAQ

This page collects common questions and recurring points of confusion about CoreRelm.

It should grow over time as real usage reveals which topics users most often stumble on.

General questions

Is CoreRelm trying to hide the database?

No. CoreRelm is designed to provide structure and readability while still keeping relational behavior understandable and visible.

Are the QuickStart examples supposed to compile and run as-is?

Not necessarily. The QuickStart project is documentation-first. Its purpose is to show what usage calls may look like and to demonstrate feature shape.

What database provider does CoreRelm currently target?

CoreRelm is currently focused on MySQL 8+.

Models and mapping

What is the difference between `RelmModel` and `RelmModelClean`?

`RelmModel` includes standard metadata fields by default, while `RelmModelClean` gives you a cleaner base when you want tighter control over the model shape.

Why does my model example work conceptually but not seem migration-ready?

Because a runtime usage example and a full schema-definition example are not always the same thing. Some model examples are intended to teach call shape and mapping intent, not to be a complete migrations-ready schema.

Contexts and datasets

What is the difference between an eager context and a quick context?

An eager context initializes datasets and verifies tables up front. A quick context disables that automatic work and loads metadata as needed.

Why do I sometimes need to retrieve a data set manually?

In quick-context-style workflows, datasets may not be initialized up front, so manual retrieval becomes part of the runtime pattern.

Transactions

Should I use explicit transactions?

Yes, especially for multi-step write workflows. Explicit transaction boundaries are one of the clearest and safest defaults in CoreRelm.

When would I use auto-open transaction behavior?

When it fits the application flow, but explicit transactions are usually easier to read and troubleshoot.

Migrations

Are migrations a first-class feature in CoreRelm?

Yes. Migrations are part of the public story and deserve both narrative docs and API reference coverage.

Why separate generating from applying migrations?

Because they answer different questions. Generating creates the artifact; applying executes it against a database.

What is schema drift?

Schema drift is the mismatch between the intended schema state and the actual database state.

Troubleshooting questions to add over time

This page should eventually collect real-world answers for issues such as:

- connection string problems
- foreign-key confusion
- migrations validation failures
- unexpected loader behavior
- result-shape misunderstandings
- DI lifetime or integration confusion

API Reference Guide

This page explains how to use the generated API reference effectively.

The API reference is valuable, but it works best when the reader knows what kind of question it is meant to answer.

What the API reference is for

The generated API docs are best for questions such as:

- what type or member exists?
- what namespace is it in?
- what is the exact public surface?
- what related types exist nearby?
- what overloads or members should I inspect next?

The API reference is not a full replacement for narrative docs. It is the detailed reference layer.

When to use the guide versus the API docs

A useful rule of thumb is:

- use the guide when you want explanation, workflow, and recommended usage
- use the API reference when you want exact type/member detail

Both are important, but they serve different purposes.

Major API areas to know

A useful way to approach the API reference is by major feature area.

Core runtime

Look here for types such as:

- `RelmContext`
- `RelmDataSet<T>`
- `RelmModel`
- `RelmModelClean`

Options and configuration

Look here for types such as:

- `RelmContextOptions`

- `RelmContextOptionsBuilder`

Helpers and extensions

Look here for utility surfaces such as:

- `RelmHelper`
- extension classes
- service-collection integration surfaces

Schema and attributes

Look here for mapping and schema-related types such as:

- `RelmTable`
- `RelmColumn`
- `RelmKey`
- `RelmForeignKey`
- other schema-oriented attribute types

Migrations

Look here for migration-related types such as:

- migration tooling
- model-set parsing and resolution
- planning and rendering types
- apply and validation result types

How to read the API docs efficiently

A good default approach is:

1. start from the guide page for the concept
2. jump to the API docs for exact type/member details
3. move sideways to related types in the same namespace or feature area
4. return to the guide if you need workflow context

Use the API docs to confirm, not to infer everything

The API docs are strongest when used to confirm details, such as:

- method names
- type relationships
- overloads

- namespaces
- publicly exposed feature areas

They are weaker as a standalone substitute for conceptual documentation.

Design guidance

A good default approach is:

- start with the guide for concepts
- use the API docs for precision
- treat the API reference as a map of the public surface, not as the only learning path

Where to go next

After this page, the best next step is QuickStart Examples Guide.

That page explains how to use the QuickStart project as a companion to both the guide and the API docs.

QuickStart Examples Guide

This page explains how to use the QuickStart project effectively.

The QuickStart project is a documentation-first examples project. Its purpose is to show what usage calls may look like across the public CoreRelm feature surface.

What the QuickStart project is for

Use `examples/CoreRelm.QuickStart` when you want to see:

- the general shape of an API call
- how a feature is typically expressed
- how related feature areas fit together
- where to look next in the guide or API reference

The QuickStart project is a usage map, not a polished end-to-end application.

What the QuickStart project is not

The QuickStart project is not primarily intended to be:

- a production-ready sample app
- a guarantee that every example compiles and runs as-is
- the only source of conceptual documentation

That distinction is important, because it changes how the examples should be read.

How to use QuickStart together with the guide

A good default approach is:

1. read the guide page for the concept
2. open the related QuickStart example file
3. inspect the API call shape
4. use the API reference for exact type/member details if needed

This three-layer workflow is often the fastest way to learn the library.

Major example areas

The QuickStart inventory covers a broad set of feature areas, including:

- attributes
- connections

- contexts
- CRUD and data-access shapes
- data loaders
- DTOs
- foreign keys
- identity helpers
- bulk writing
- model reset and model property patterns

That breadth is one of the reasons the QuickStart project is worth documenting explicitly.

Example interpretation guidance

When reading a QuickStart example, ask:

- what feature is this example trying to demonstrate?
- what call shape is it showing?
- is this a runtime example, a schema example, or both?
- should I go to the guide for explanation or the API docs for precision?

Those questions help keep the project useful instead of confusing.

Suggested mapping approach

Over time, this page should grow into a feature-to-example map, such as:

- **Introduction / Getting Started**
  - Quick start patterns in `Program.cs`
- **Core Concepts**
  - context, options, and model examples
- **Working with Data**
  - CRUD, result shapes, relationships, DTOs, loaders, bulk writing
- **Advanced Topics**
  - migrations, helpers, integration, and advanced workflows

Design guidance

A good default approach is:

- use QuickStart to understand API shape
- use the guide to understand concepts and recommendations
- use the API reference to confirm exact members and types

Where to go next

After this page, the best next step is Architecture Notes.

Architecture Notes

This page explains the high-level architectural shape of CoreRelm.

It is not meant to be a deep internal design specification. Instead, it should help readers understand the main layers of the project and how the public pieces fit together.

High-level architecture

A useful way to think about CoreRelm is as several connected layers:

- model and attribute metadata
- runtime context and dataset workflows
- options and configuration
- helper and extension surfaces
- schema and migrations tooling
- examples, tests, and generated API docs

Each layer supports a different kind of developer task.

Public surface versus internal machinery

One of the useful distinctions in any library is the difference between:

- what users are expected to work with directly
- what exists mainly to support those public features behind the scenes

The CoreRelm guide should generally focus on the public mental model first, then expose deeper internals only when they help users reason about advanced behavior.

Metadata-centered design

A major architectural theme in CoreRelm is that metadata stays close to the model through attributes.

That helps keep relational intent readable and discoverable without forcing everything into a separate configuration system.

Runtime-centered design

Another important theme is that runtime behavior gathers around the context.

This makes the context the central place where:

- configuration is applied
- data sets are accessed

- transactions are controlled
- helper operations are performed

Schema and migrations as a real subsystem

Migrations are not just an optional sidecar feature. They represent a real subsystem in the project architecture.

That matters because it means the project has two major stories:

- runtime ORM usage
- schema evolution and migration workflows

Docs, examples, tests, and API reference as architecture support

The project also has a broader support architecture for learning and maintenance:

- guide pages for explanation
- QuickStart examples for usage shape
- tests for behavior verification
- generated API docs for precision

That is worth documenting because it tells contributors where each kind of documentation responsibility belongs.

Extension points and boundaries

Over time, this page should become more specific about:

- intended extension points
- provider-specific assumptions
- what is considered stable public behavior
- what is more internal or subject to change

Design guidance

A good default approach is:

- explain architecture in terms of responsibilities, not only file layout
- keep the public mental model front and center
- introduce deeper implementation detail only when it improves understanding

Where to go next

After this page, the best next steps are:

1. Contributing
2. Roadmap

Contributing

This page explains how to contribute to CoreRelm.

As the project grows, it is helpful to make contribution expectations clear for code, tests, docs, examples, and broader project maintenance.

General contribution mindset

A good contribution should aim to improve one or more of the following:

- clarity
- correctness
- maintainability
- documentation quality
- test coverage
- user experience for library consumers

Core expectations

A good default expectation is:

- keep changes focused
- prefer clear and explicit code
- add or update tests when behavior changes
- update documentation when public behavior changes
- keep examples aligned with the public surface

Documentation contributions

If a public feature changes, documentation should usually be updated in more than one place.

That may include:

- the guide pages
- the QuickStart examples
- the README
- generated API docs or API-doc inputs
- tests that serve as behavioral evidence

This matters because different documentation layers serve different purposes.

Example contributions

QuickStart examples should stay aligned with the real public surface.

Because the QuickStart project is documentation-first, example contributions should prioritize:

- clear usage shape
- focused feature demonstration
- consistency with the guide
- consistency with actual public types and members

Code contributions

Code contributions should keep CoreRelm's design style in mind:

- explicitness over magic
- readable relational behavior
- visible transaction and configuration boundaries
- practical, understandable APIs

Test contributions

Tests are especially important for:

- migrations behavior
- parsing and planning logic
- relationship behavior
- configuration and options behavior
- public feature regressions

A good contribution often includes the tests needed to make the change trustworthy.

Documentation-first improvements

Not every valuable contribution needs to be a code change.

Useful contributions can also include:

- clarifying docs wording
- mapping examples to guide pages
- improving troubleshooting coverage
- refining architecture notes
- improving contribution guidance itself

Design guidance

A good default approach is:

- make the smallest clear change that solves the problem

- support public changes with tests and docs
- keep the project easier to understand after the change than before it

Where to go next

After this page, the next best step is Roadmap.

Roadmap

This page describes the current documentation and project-direction priorities at a high level.

It should evolve over time as the project matures.

Current documentation priorities

The current documentation direction strongly suggests a few immediate priorities:

- complete the narrative guide structure
- keep the QuickStart project aligned with the real public surface
- strengthen migrations coverage as a first-class docs area
- improve troubleshooting and FAQ coverage
- keep README, guide pages, examples, and API docs aligned

Current product-direction themes

At a high level, the current project direction emphasizes:

- a lightweight, explicit ORM model
- strong readability around contexts, models, and datasets
- MySQL 8+ support
- migrations and schema tooling as a meaningful subsystem
- documentation that serves both beginners and advanced users

Suggested near-term goals

Useful near-term goals include:

- finish and refine all guide pages
- add a richer feature-to-QuickStart mapping
- deepen migrations and schema examples
- improve DI and integration guidance
- expand troubleshooting coverage based on real user questions

Suggested medium-term goals

Useful medium-term goals include:

- strengthen architecture notes
- clarify extension points and stability expectations
- add more operational guidance for migrations
- continue improving narrative consistency across docs layers

Roadmap maintenance guidance

A roadmap page is most useful when it stays:

- honest
- lightweight
- regularly updated
- focused on priorities rather than promises

Design guidance

A good default approach is:

- use the roadmap to communicate direction
- avoid treating it as a rigid guarantee
- keep it aligned with actual project and docs work

Where to go next

After this page, return to the broader guide, QuickStart examples, or API reference depending on the kind of information you need next.